NOVA / WAREHOUSE

Отчёт по программному проекту

Система управления оптовым складом

Цель работы: разработать и исследовать программную модель снабжения магазинов с учётом партий, сроков годности, упаковок, поставок, уценки и списаний. Результат оценивается по движению товаров и финансовым показателям склада.

Объект описания: версия NovaPochta_r с настройками спроса, запаса и закупочной цены, графическим интерфейсом и возвратом к параметрам через кнопку «Новый сценарий».

Основания: техническое задание «Система управления оптовым складом», исходники проекта и выполненные контрольные проверки backend. В отчёте отдельно отмечены ограничения текущей реализации; полное совпадение всех функций с исходным заданием не предполагается автоматически.

Репозиторий: github.com/cloudly28-arch/NovaPochta_r. Описание учитывает подготовленные изменения файлов и не утверждает, что каждая из них уже включена в опубликованную ветку.

Содержание

1 Уточнённая постановка задачи 2

2 Диаграмма классов программной системы 4

3 Текстовые спецификации основных классов 7

4 Использованные инструментальные средства 10

5 Файловая структура программной системы 11

6 Краткая характеристика пользовательского интерфейса 12

7 Краткое описание проведённых экспериментов 13

1 Уточнённая постановка задачи

1.1 Предметная область и назначение

Оптовый склад обслуживает M торговых точек и хранит K видов продуктовых товаров. Требуется автоматизировать учёт остатков и партий, обработку заказов, формирование заявок поставщикам, уценку и списание просроченных продуктов. Для проверки решений моделируются потребление магазинов и движение поставок.

Границы исходного задания:

$$3 \le M \le 9, \qquad 12 \le K \le 20, \qquad 10 \le N \le 30,$$

где N — число моделируемых дней. Основной отчётный шаг — один день. В реализации внутри дня выделено восемь трёхчасовых интервалов продаж. Интерфейс дополнительно допускает период до 100 дней; описанные далее прогоны на 10 и 30 дней соответствуют диапазону исходного задания.

1.2 Входные данные и параметры

Для товара задаются наименование, обычная цена, срок годности и число единиц в оптовой упаковке. Для пары «магазин–товар» определяются ассортимент, остаток, вместимость и минимальный запас. Вместимость склада ограничена отдельно по каждой позиции и хранится в единицах товара; допустимое число полных упаковок получается делением на размер упаковки.

В базе подготовлены девять магазинов и двадцать позиций с фиксированными ассортиментами. При уменьшении M или K используются первые записи по ID. Отдельный редактор произвольного ассортимента в интерфейсе отсутствует.

Параметр	Диапазон	Назначение
Продолжительность	10–100 дней в UI	Горизонт эксперимента.
Магазины / товары	3–9 / 12–20	Размер моделируемой сети.
Спрос	50–150%	Масштабирование продаж и прогноза потребления.
Начальная заполненность	Авто или 0–100%	Состояние склада на старте.
Целевой резерв	Авто или 1–14 дней	Ориентир для автозапаса и пополнения.
Закупочная цена	1–100% обычной цены	Единая закупочная доля эксперимента.
Подтверждение заявок	Ручное / автоматическое	Участие пользователя в поставках.

1.3 Основные правила обработки

1. Магазины расходуют остатки в соответствии со случайным потреблением. Позиция заказа возникает при нехватке относительно минимума либо наличии подходящей скидочной партии. В SQL-пути создания многопозиционного заказа предусмотрен запрет второго заказа магазина за тот же день.
2. Запрос ограничивается свободным местом магазина и формируется в целых упаковках. Склад допускает частичное выполнение.

3. Партии отбираются по ближайшему сроку истечения (FEFO). Выделяются только партии, годные на плановый день доставки.
4. Выделение уменьшает доступный остаток и создаёт резерв заказа. После доставки увеличивается остаток магазина, заказ завершается.
5. При расчёте пополнения учитываются прогноз спроса, нехватка магазинов, невыполненные позиции заказов, сроки годности, вместимость и активные заявки поставщику. Задержка поставки выбирается от одного до пяти дней и отсчитывается от подтверждения.
6. За два дня до истечения автоматическая уценка повышает скидку до 15%, за один день — до 30%, если текущая скидка ниже. Ручная уценка подходящих партий доступна в последние три дня годности.
7. Партия годна при $t < E_b$ и подлежит списанию доступного остатка при $t \geq E_b$, где E_b — день истечения её срока.

Для базового спроса за интервал выбирается одна или две единицы, в среднем 12 за день при достаточном наличии. Интенсивность u даёт ожидаемое дневное потребление $12u/100$; дробное количество округляется вероятностно. Реальная продажа не превышает остаток магазина.

1.4 Выходная информация и экономика

Система предоставляет остатки, партии, скидки, заказы, резервы, поставки, списания, выручку и итоговые показатели по товарам и дням. Финансовая граница — склад как оптовый продавец магазинам. Выручка признаётся только по завершённым доставкам.

При обычной цене единицы P в копейках, закупочной доле k и скидке d:

$$c = \left\lfloor \frac{Pk + 50}{100} \right\rfloor, \qquad p = \left\lfloor \frac{P(100 - d) + 50}{100} \right\rfloor.$$

Сначала округляется цена единицы, затем она умножается на количество. Если $\mathcal{D}$ — завершённые выделения партий, $\mathcal{W}$ — списания, то

$$R = \sum_{\ell \in \mathcal{D}} q_\ell p_\ell, \quad C = \sum_{\ell \in \mathcal{D}} q_\ell c_\ell, \quad W = \sum_{w \in \mathcal{W}} q_w c_w, \quad \boxed{G = R - C - W}.$$

Уценка уже включена в R и второй раз не вычитается. Непроданный товар остаётся запасом: вся полученная поставка не списывается немедленно в расходы. Результат G не равен денежному потоку или чистой прибыли, так как платежи, перевозки, зарплаты, хранение и налоги не моделируются.

1.5 Отличия текущей версии от полного задания

Вес выбора позиции от процента скидки рассчитывается, но не используется при случайном выборе: включаются все подходящие кандидаты. Отдельные диапазоны случайных величин не редактируются пользователем полностью. Исходный запрос уже приведён к упаковкам. Подтверждение поставщиков и ручная уценка реализованы, но отдельного общего утверждения плана перевозок и предложений автоматической уценки нет. Эти ограничения не подменяются описанием отсутствующих возможностей.

2 Диаграмма классов программной системы

2.1 Основные классы графического сценария

На рисунке 1 показаны классы основного пути выполнения. Закрашенный ромб означает композицию: объект хранится как поле владельца. Стрелка сплошной линии обозначает обращение через невладеющий указатель, пунктир — зависимость. Отношения показаны на уровне типов; список полей и операций сокращён до существенных.

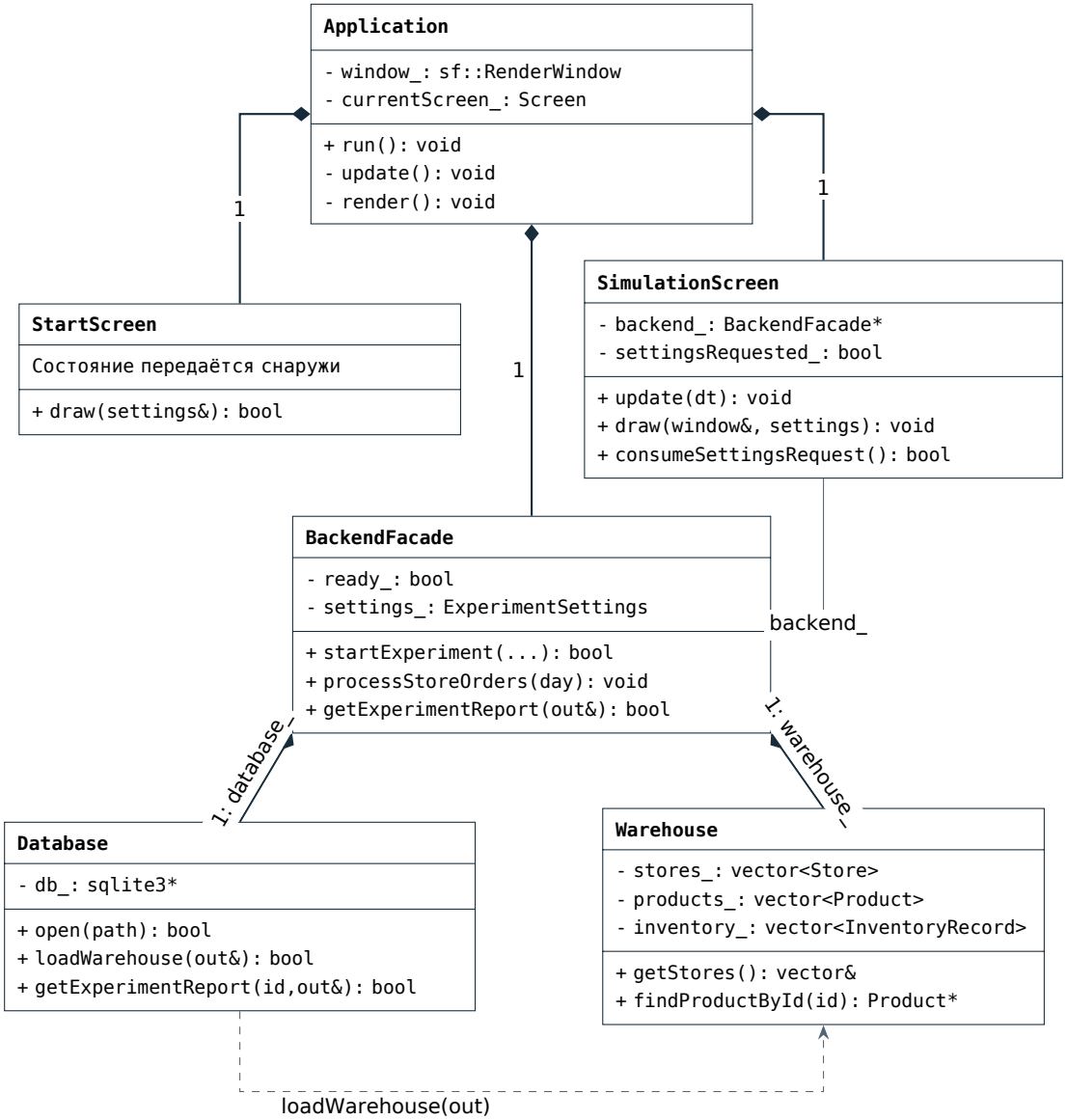


Рис. 1: Основные классы приложения и владение объектами

2.2 Объекты предметной области

`Warehouse` хранит коллекции объектов по значению. Все коллекции имеют кратность $0..*$. Связи товара с остатком и позицией заказа, а также магазина с заказом реализованы идентификаторами. Для текущих партий и финансовых журналов используются также структуры строк БД, описанные в разделе 3.

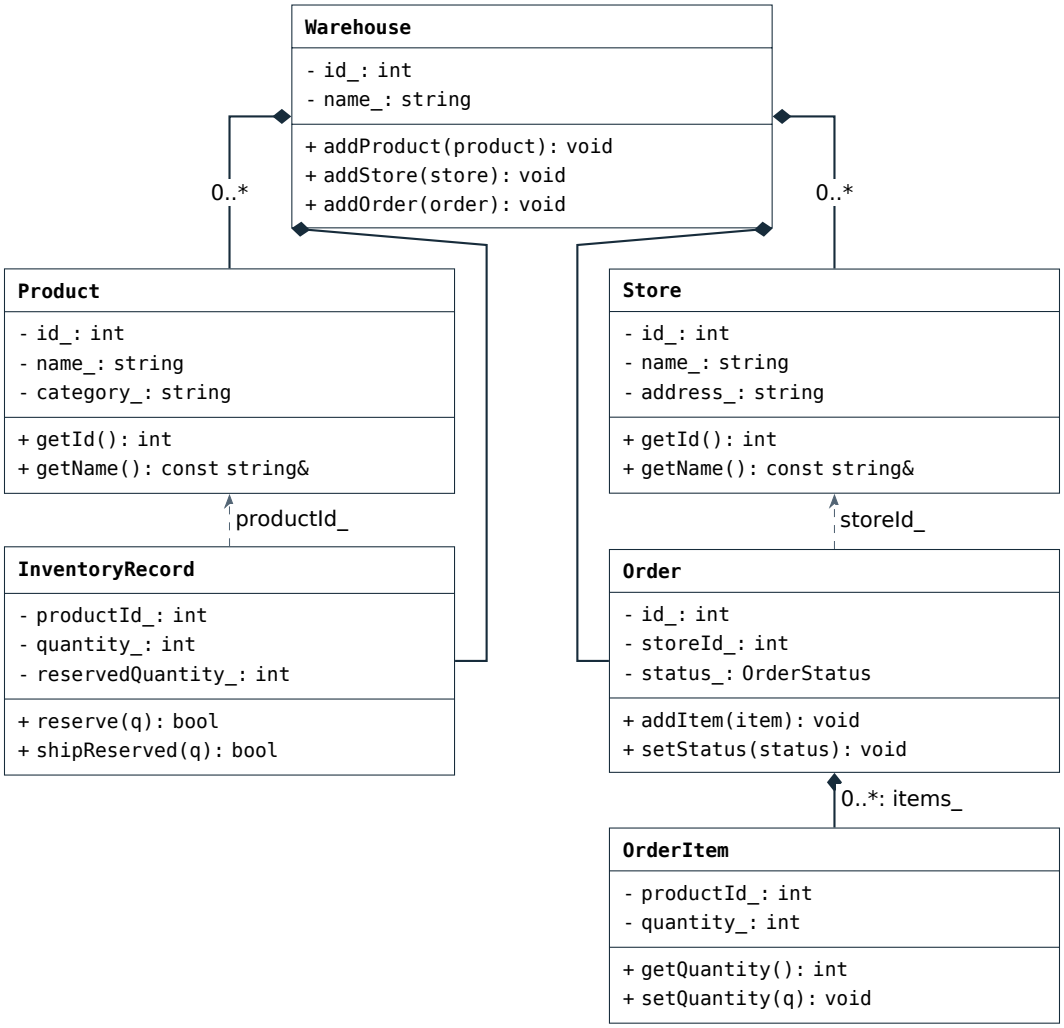


Рис. 2: Основные объекты учёта и их связи

Дополнительно в `Warehouse` предусмотрены коллекции `Worker`, `StorageCell`, `WarehouseZone`, `WarehouseTask` и `Vehicle`. Их наличие не означает автоматическое использование всех сущностей графическим сценарием.

2.3 Объектный сервисный слой

Сервисы на рисунке 3 работают со ссылками на `Warehouse`. `OrderService` дополнительно использует `InventoryService`. У первых двух сервисов есть необязательный указатель `Database*`. Наследование между показанными классами отсутствует.

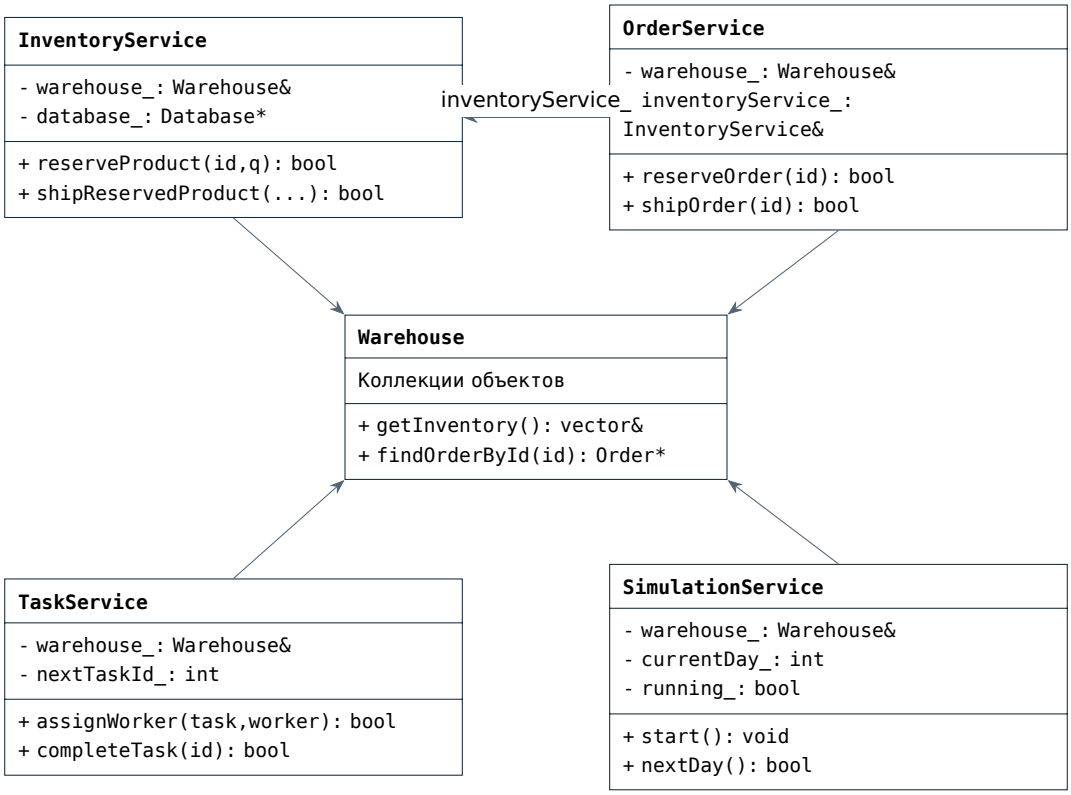


Рис. 3: Сервисы объектного слоя

Основной графический путь обработки идёт через `SimulationScreen -> BackendFacade -> Database`. Объектные сервисы также представлены в отдельной консольной программе; их нельзя считать единственным исполнительным механизмом карты. Доменный `OrderStatus` и строковые статусы SQL-заказов относятся к разным представлениям и не совпадают полностью.

3 Текстовые спецификации основных классов

В спецификациях приведены назначение, существенное состояние, публичные операции и ограничения. Типы параметров в диаграммах сокращены; точные объявления находятся в соответствующих заголовках.

3.1 Application

Назначение: управление жизненным циклом графического приложения. Владеет окном, часами, фасадом backend, экранами и пользовательскими настройками.

Интерфейс: конструктор, деструктор и `run()`. Внутренние операции — `processEvents()`, `update()`, `render()`.

Поведение: конструктор подключает ImGui, шрифт, ресурсы и базу; `run()` выполняет цикл событий, изменения состояния и отрисовки. При подтверждении параметров вызывается `BackendFacade::startExperiment`. Переход к карте происходит только после успешной подготовки эксперимента. Запрос `consumeSettingsRequest()` возвращает приложение к экрану настройки.

Условия: backend существует всё время жизни использующего его экрана. При невозможности начальной подготовки приложение закрывает окно; ошибка нового эксперимента выводится через окно сообщения.

3.2 StartScreen

Назначение: ввод параметров сценария средствами ImGui.

Интерфейс: `bool draw(SimulationSettings& settings)`. Изменяемые значения передаются по ссылке; собственная копия настроек в классе не хранится.

Предусловие: создан контекст ImGui и начат кадр. **Постусловие:** значения формы записаны в `settings`; возвращено `true` только при нажатии кнопки запуска.

Особенности: галочка ручного заполнения включает значение 50%, снятие задаёт -1 для автоматического режима. Аналогично галочка резерва включает 5 дней, снятие задаёт 0. Ползунки не заменяют проверку параметров backend.

3.3 SimulationScreen

Назначение: графический ход эксперимента, карта, панели, транспорт и представление статистики.

Состояние: текущий день и таймер, скорость, число обработанных интервалов продаж, состояние `Stopped/Running/Paused/Finished`, выбор объекта, визуальные машины, снимки финансов и `BackendFacade*`.

Интерфейс: `configure(settings)`, `update(dt)`, `draw(window,settings)`, `setBackend(pointer)`, `consumeSettingsRequest()` и установка путей ресурсов.

Контракты: `configure` сбрасывает время, выбор, машины, кэш сцены и историю; начальное состояние — `Stopped`. `update` проводит продажи только для ещё не обработанных интервалов. Ошибка интервала переводит моделирование в паузу. Запрос нового сценария устанавливает паузу; `consumeSettingsRequest` возвращает флаг и сразу сбрасывает его, поэтому переход выполняется однократно.

Ограничение: прибытие визуального транспорта вызывает операции доставки

backend. Анимация пока не полностью отделена от событий модели.

3.4 ExperimentSettings и SimulationSettings

Назначение: передача конфигурации без зависимости от SQL или графики.

`ExperimentSettings` содержит четыре целочисленных поля: `initialFillPercent`, `demandPercent`, `stockCoverDays`, `purchaseCostPercent`. Значения по умолчанию: -1, 100, 0 и 70. Метод `isValid()` проверяет диапазоны соответственно $[-1, 100]$, $[50, 150]$, $[0, 14]$, $[1, 100]$.

`SimulationSettings` добавляет `days`, `stores`, `products` и `automaticSupplierApproval`. По умолчанию используются 20 дней, 9 магазинов, 20 товаров и ручное подтверждение. Дни и размеры сценария проверяются отдельно от параметров модели.

3.5 BackendFacade

Назначение: прикладные операции склада и согласование шагов эксперимента. Владеет `Database`, `Warehouse` и копией параметров.

Основные операции:

- `initialize(path)` открывает базу, загружает склад и параметры; успех устанавливает `ready_`.
- `startExperiment(path,schema,M,K,settings)` проверяет значения и подготавливает новый эксперимент; при неуспехе восстанавливает прежние поля конфигурации фасада.
- `simulateStoreSales(day,interval)` уменьшает остатки магазинов в транзакции; допустимы день > 0 и интервал 1–8.
- `processStoreOrders(day)` создаёт и пополняет резервы заказов, ограничивая количество пригодным запасом, упаковками и местом магазина.
- `processSupplierRequests(day)` оценивает пополнение с учётом активных заявок; `approveSupplierRequest(id,day)` подтверждает созданную заявку.
- `completeSupplierRequest(id,receivedDay)` согласованно меняет остаток, партию и статус; повторное завершение уже доставленной заявки успешно.
- `getExperimentReport(out)` передаёт запрос агрегированного отчёта.

Ошибки: булевы методы сообщают успех явно; пояснение доступно через `getLastError()`. Часть обработчиков возвращает `void`, поэтому вызывающий код не всегда автоматически останавливает весь эксперимент.

3.6 Database

Назначение: доступ к SQLite, транзакции, журналы партий и финансовая агрегация. Состояние — указатель `sqlite3*`; закрытое соединение обозначается `nullptr`. Копирование объекта запрещено; деструктор закрывает соединение.

Интерфейс: открытие и закрытие, выполнение SQL, чтение остатков, создание заказов и заявок, выделение партий, списание, настройки и отчёт. Пара `getExperimentSettings/setExperimentSettings` читает и сохраняет конфигурацию.

Контракт выделения: `changeWarehouseQuantity(...)` при отрицательном изменении проверяет пригодность партий на нужный день, уменьшает партии, фиксирует цены в `warehouse_allocations` и меняет доступный остаток. При ошибке точка сохранения позволяет откатить связанные действия.

Контракт отчёта: `getExperimentReport(id,out)` учитывает для выручки и себестоимости продаж только заказы `Completed`. Журналы сначала агрегируются отдельно по товарам, затем соединяются, чтобы не умножить суммы при связях «один ко многим». Себестоимость списаний рассчитывается по закупочной доле; `warehouse_writeoffs.loss_cents` сохраняется как обычная, справочная стоимость.

3.7 Warehouse

Назначение: агрегат объектов предметной области в памяти. Содержит идентификатор, название, адрес и коллекции магазинов, товаров, запасов, заказов, сотрудников, заданий, транспорта, зон и ячеек.

Интерфейс: методы `add...`, `get...`, `find...ById`. Методы поиска возвращают указатель на найденный объект либо `nullptr`.

Ограничение: загруженное состояние `Warehouse` не является автоматически синхронизированной копией всех SQL-таблиц. В графическом пути актуальные остатки обычно запрашиваются через `Database`.

3.8 InventoryRecord

Назначение: объектная запись количества товара и резерва. Содержит ID товара и ячейки, количество, резерв, номер партии и сроковую строку.

Инвариант: $0 \le reservedQuantity \le quantity$; доступно $quantity - reservedQuantity$. `reserve(q)` допустим при положительном количестве и достаточном наличии. `releaseReservation(q)` снимает существующий резерв, `shipReserved(q)` одновременно уменьшает количество и резерв. Конструктор с параметрами отклоняет недопустимые количества и идентификаторы.

Объектная запись и SQL-строка `warehouse_inventory` имеют разные способы представления резерва: в графическом SQL-пути он уже исключён из доступного остатка.

3.9 Order и OrderItem

`Order` хранит ID, магазин, `OrderStatus`, дату и коллекцию позиций. `addItem` объединяет количества одинакового товара, `removeItemByProductId` удаляет позицию, `getTotalItemCount` суммирует количества. `OrderItem` задаёт ID товара и положительное количество; конструктор с параметрами и изменение количества контролируют допустимость значений.

Это объектный слой. В графическом SQL-пути применяются структуры `StoreOrderRow/StoreOrderInfo`, а позиции имеют отдельно `requestedQuantity` и `allocatedQuantity`. Статус `Allocated` может соответствовать частичному выделению. Завершение заказа не означает равенства запрошенного и доставленного.

3.10 Сервисы и структуры передачи данных

`InventoryService` принимает товар, резервирует, освобождает резерв, отгружает и списывает объектные остатки. `OrderService` согласует создание, резервирование, отмену и отгрузку объектного заказа; при неуспехе резервирования освобождает уже созданные резервы. `TaskService` создаёт задания подбора и назначает свободных сотрудников. `SimulationService` хранит простой счётчик дней и признак работы; он не является единственным контроллером времени графического

приложения.

Структуры `WarehouseBatchRow`, `StoreStockRow`, `WarehouseStockRow` и другие `Row` передают результаты SQL-запросов. Структуры `Info` передают снимки фасада в интерфейс. `ExperimentReport` содержит параметры, итоги и вектор `ExperimentProductResult`; денежные суммы представлены `long long` в копейках. Изменение полученного снимка само по себе не записывает данные в базу.

4 Использованные инструментальные средства

Средство	Применение
C++17 / C++20	Backend / графическое приложение соответственно.
GCC, MSYS2 UCRT64	Компиляция под Windows; команды проекта ориентированы на `C:/msys64/ucrt64`.
CMake, Ninja	Конфигурация, сборка библиотеки, приложений и теста.
SFML 3	Окно, события, текстуры, фигуры, карта и транспорт. В пользовательском логе обнаружена версия 3.0.2.
Dear ImGui 1.91.9	Элементы управления, таблицы, панели и подсказки; версия закреплена в CMake.
ImGui-SFML 3.0	Связь ImGui с окном и событиями SFML.
SQLite	Встроенная база данных без отдельного сервера.
Visual Studio Code	В репозитории имеются настройки `.vscode` для CMake. Фактический выбор IDE разработчиком отдельно не зафиксирован; сборка не зависит от конкретного редактора.
PowerShell / CMD	Настройка окружения, сборка и запуск на Windows.
Git / GitHub	Версионирование проекта и получение закреплённых зависимостей.
clang-format	Оформление на основе Google Style с конфигурацией в корне.
XeLaTeX, TikZ	Подготовка отчёта и векторных UML-диаграмм.

Фасад и интерфейс находятся в одном процессе, поэтому REST/TCP API не требуется. При первом конфигурировании ImGui и ImGui-SFML получаются через CMake FetchContent; SFML и SQLite должны быть доступны как установленные зависимости с заголовками и библиотеками.

Пример сборки из PowerShell в корне проекта:

```
$env:Path = "C:\msys64\ucrt64\bin;" + $env:Path
cmake -S front -B front/build-local -G Ninja
↪  -DCMAKE_CXX_COMPILER=C:/msys64/ucrt64/bin/g++.exe -DCMAKE_PREFIX_PATH=C:/msys64/ucrt64
↪  -DSFML_DIR=C:/msys64/ucrt64/lib/cmake/SFML
cmake --build front/build-local -j 4
.\front\build-local\NovaPoshtaWarehouseFrontend.exe
```

5 Файловая структура программной системы

Путь	Назначение
`front/CMakeLists.txt`	Сборка frontend, подключение зависимостей и backend, копирование ресурсов.
`front/include/Application.h`	Состав и интерфейс приложения.
`front/include/ui/StartScreen.h`	Экран параметров и `SimulationSettings`.
`front/include/ui/SimulationScreen.h`	Состояния, визуальные объекты, операции экрана моделирования.
`front/src/main.cpp`	Точка входа графического приложения.
`front/src/Application.cpp`	Цикл событий, смена экранов, инициализация ресурсов, базы и темы.
`front/src/ui/StartScreen.cpp`	Ползунки, галочки, подсказки и запуск сценария.
`front/src/ui/SimulationScreen.cpp`	Время, карта, перевозки, панели, кнопка нового сценария и отчёт.
`front/assets/textures/`	Изображения склада, магазинов, поставщика и машин.
`nova_poshta_warehouse_backend/CMakeLists.txt`	Библиотека backend, консольная программа и необязательные тесты.
`nova_poshta_warehouse_backend/include/domain/`	Заголовки объектов предметной области.
`nova_poshta_warehouse_backend/include/domain/ExperimentSettings.h`	Настройки модели и их проверка.
`nova_poshta_warehouse_backend/include/application/BackendFacade.h`	Прикладной интерфейс склада для frontend.
`nova_poshta_warehouse_backend/include/persistence/Database.h`	SQLite-операции, структуры строк и результаты отчёта.
`nova_poshta_warehouse_backend/include/services/`	Интерфейсы объектных сервисов.
`nova_poshta_warehouse_backend/src/domain/`	Реализация доменных классов.
`nova_poshta_warehouse_backend/src/application/BackendFacade.cpp`	Правила операций эксперимента.
`nova_poshta_warehouse_backend/src/persistence/Database.cpp`	SQL, транзакции, партии и финансовая агрегация.
`nova_poshta_warehouse_backend/src/services/`	Реализация сервисов.
`nova_poshta_warehouse_backend/src/main.cpp`	Отдельная консольная точка входа.
`nova_poshta_warehouse_backend/database/schema.sql`	Создание таблиц, представлений и исходных данных.
`nova_poshta_warehouse_backend/tests/experiment_settings_test.cpp`	Регрессионные проверки параметров и модели.

Путь	Назначение
`.clang-format, .vscode/`	Правила форматирования и настройки редактора.

Данные во время выполнения. Приложение использует каталог исполняемого файла. В нём ожидаются `assets/`, `database/schema.sql` и рабочий файл `database/nova_poshta_warehouse.db`. Данные хранятся в таблицах справочников, остатков, партий, заказов, выделений, заявок и списаний. Таблица `experiment_settings` добавляется автоматически, если отсутствует.

Создание эксперимента. SQL-схема сначала применяется к временной базе в памяти, затем настраиваются магазины, товары и начальные остатки, после чего подготовленная база копируется в рабочий файл. Новый эксперимент заменяет данные предыдущего; это не система нескольких независимых сохранений.

6 Краткая характеристика пользовательского интерфейса

6.1 Экран параметров

Используется тёмная тема с бирюзовыми акцентами и двумя панелями: «Сценарий» и «Склад и экономика». В первой задаются длительность, количество магазинов и товаров, подтверждение поставок; во второй — спрос, закупочная доля и ручные параметры запаса. Краткие подписи сопровождаются подсказками при наведении. Галочки раскрывают ручные ползунки. На узком экране панели размещаются вертикально с прокруткой.

Кнопка запуска создаёт эксперимент и открывает карту. Течение времени включается отдельной кнопкой «Старт» на карте.

6.2 Экран моделирования

Карта отображает склад, магазины, поставщика, дороги и машины. Выбор объекта мышью открывает его данные. Верхняя панель показывает день, статус и прогресс, нижняя — старт, паузу, сброс и скорость. Складские панели предоставляют партии, сроки, скидки, списания и потребности; панель поставщика позволяет подтверждать заявки.

Добавленная кнопка «Новый сценарий» приостанавливает моделирование и возвращает к форме с последними значениями параметров. База пересоздаётся только после запуска нового эксперимента. «Сброс» сохраняет назначение повтора текущих параметров с первого дня.

6.3 Отчёт по периоду

Денежные показатели представлены компактной таблицей: выручка, закупочная стоимость доставок, потери от списаний и результат. Названия выровнены слева, суммы — справа, тысячи разделены пробелами; например, 1 496 276,50 руб. Подробности расчёта и параметры раскрываются отдельно. Доступны график по дням и таблица товаров.

График показывает выручку и себестоимость списаний, а не чистую прибыль.

Суммы выводятся в рублях, но рассчитываются в целых копейках. Связь со складскими операциями обеспечивается фасадом; UI не изменяет формулы финансового отчёта самостоятельно.

7 Краткое описание проведённых экспериментов

7.1 Методика и границы проверки

Проверки выполнены для backend с исходными данными `schema.sql`. Использовался регрессионный тест `experiment_settings_test.cpp`: 9 магазинов и 20 товаров, отдельная тестовая база, фиксированные правила инициализации генераторов по дню и интервалу. В каждом дневном прогоне проверялись неотрицательные остатки, соблюдение вместимости, равенство суммы доступных партий доступному складскому остатку и согласованность итогов по товарам.

Это headless-прогоны: доставка вызывается календарными операциями теста, а не движением машин. Поэтому успешный backend-тест не доказывает полную эквивалентность графического сценария при всех скоростях.

7.2 Проверка параметров и реакции модели

Проверка	Условия	Наблюдаемый результат
Начальная заполненность	0%, 50%, 100% и авто	Количества соответствуют формуле заполнения, кратны упаковкам и не превышают вместимость.
Хранение настроек	Спрос 150%, резерв 4 дня, закупка 63%	Значения сохранены после повторного открытия базы и сброса эксперимента.
Границы параметров	Заполненность 101%, спрос 49%, резерв 15, закупка 0% проверялись отдельно	Некорректные конфигурации отвергнуты.
Интенсивность спроса	50 / 100 / 150%, один день без пополнения магазинов	Продано соответственно 162 / 323 / 485 единиц.
Дни резерва	2 и 7 дней, автоматический старт	Суммарный начальный запас 844 и 2276 единиц; заявки при пустом старте также увеличиваются.
Старая база	Удалена таблица настроек перед повторным открытием	Таблица восстановлена, значения по умолчанию применены, остатки магазинов сохранены.

Числа 162/323/485 и 844/2276 относятся к конкретному контрольному набору и генераторам. Их нельзя трактовать как универсальную оценку спроса или доказательство оптимальности выбранного резерва.

7.3 Проверка финансовых расчётов

В тестовую базу добавлены две завершённые доставки из одной партии на 4 и 6 единиц, незавершённый резерв на 3 единицы, списание 2 единиц и полученная поставка 12 единиц. Обычная цена — 10001 коп., фактическая — 7001 коп., закупочная доля — 70%. Полученная единичная себестоимость: 7001 коп.

Показатель	Контрольное значение
Завершённая доставка	10 единиц
Выручка	70010 коп.
Себестоимость доставленного	70010 коп.
Себестоимость списаний	14002 коп.
Справочное снижение выручки	30000 коп.
Получено от поставщика	12 единиц
Результат после себестоимости и списаний	-14002 коп.

Резерв не попал в выручку и себестоимость продаж; вся полученная поставка не была немедленно списана в расходы. Снижение выручки не вычтено повторно. Тем самым проверены округление до копейки, исключение незавершённых доставок и отсутствие двойного учёта скидки.

7.4 Длительные прогоны

Дней	Настройки	Итог проверки
10	Автоматические запас и резерв, спрос 100%, закупка 70%	Прогон завершён; инварианты остатков и суммы отчёта соблюдены.
30	Пустой склад, спрос 150%, резерв 2 дня, закупка 70%	Прогон завершён; проверен сценарий восстановления после дефицитного старта.
30	Заполненность 100%, спрос 50%, резерв 14 дней, закупка 100%	Прогон завершён; ограничения и согласованность учёта сохранены.

В этих прогонах не фиксировалась сравнительная таблица конечной прибыли для поиска оптимальной стратегии. Поэтому вывод ограничен корректностью учёта и работоспособностью проверенных сценариев.

7.5 Проверка интерфейса и выводы

Для изменённых экранов проверены синтаксис и форматирование. Компоновка стартового экрана рассматривалась при 1280×720 и 800×720; денежная таблица проверена с крупными суммами. Полный интерактивный запуск последней графической версии под Windows в среде подготовки отчёта не выполнялся. Переход к новому сценарию проверен по цепочке флага запроса, смены экрана и повторной инициализации; его сквозной пользовательский прогон остаётся отдельной проверкой.

Проверки подтверждают реакцию модели на спрос и резерв и согласованность финансовых расчётов в перечисленных сценариях.